本章路線

這章要回答的核心問題是：如何用可強制執行的資料結構，限制主體對物件的能力，並在撤銷後維持可證明的一致性？

前置知識。 mode bit、access control 基礎、graph 與 matrix。

- 區分 subject/object/domain/right
- 比較 ACL 與 capability
- 分析 revocation 對已開 handle 的影響

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

保護的問題是「誰可對哪個物件做什麼」

主體（subject）是發出請求的執行者，例如行程；**保護域（domain）**是執行者當下可行使的物件與權限集合。object 是資源，right 是可執行的操作。同一主體經授權切換 domain 後，能行使的權限可以改變；不能把 subject 與 domain 當成同一個概念。

政策例：「助教可讀作業，只有任課教師可改成績」；ACL、capability 與受保護的系統呼叫是實施政策的機制。完整仲介（complete mediation）要求每次使用都受到有效授權約束。若在 open 時檢查 ACL、往後以受保護 handle 的權限檢查 read/write，也必須明定 ACL 變動後既有 handle 的撤銷語義。

reference monitor 必須防止繞過、保護自身狀態，並足夠小而可檢查。硬體 mode 與位址空間隔離是部分支撐，不能單獨證明應用政策正確。安全（security）還涉及認證、攻擊、故障、稽核與信任；難猜的路徑不是存取控制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

基底與界限：政策如何交給硬體執行



機制：比較 base ≤ address < base + limit；不符合時觸發 fault。

政策：作業系統決定行程可存取的範圍，並安全設定 base 和 limit。

本例以 byte 為單位，檢查實體位址是否落在 `[base, base + limit)`；假設加法不溢位。作業系統決定行程可使用的範圍並保護設定；硬體比較位址並在越界時觸發 fault。實際硬體也可能先檢查相對位址再做重定位，不能把本圖當成所有架構的實作。

最小權限同時限制範圍與時間

備份工作只需讀 `/course/submissions`、寫備份目標，不應同時有修改成績或載入核心模組的權限。若整個程式一直用全權帳號，解析一個惡意檔案的錯誤就可能取得所有權限。拆成低權解析器與小型授權服務，服務仍須驗參數與物件範圍。

權限集合 R(t) 要包含當下必要權限 N(t)，並盡量縮小 R(t)−N(t)。這不是單純「權限數量越少越好」：只有 read 但可讀所有學生個資，仍可能比可寫單一暫存檔風險大。細粒度 scope、持續時間與可委派性都影響損害界。

fail-safe defaults：未找到明確授權就拒絕。稽核記錄 subject、object、operation、decision、時間與關聯 id；有 log 不代表攻擊被阻止。separation of duty 可要求兩角色批准成績發布，但若同一使用者能自行同時啟用兩角色，政策仍可能失效。

Domain 是權限集合，不等於 CPU mode

D={⟨F1,{read}⟩,⟨F2,{read,write}⟩}。同一 user 的兩個 process 可以有不同 domain；同一 process 的受保護呼叫也可能切 domain。user/kernel 兩個 mode 無法單獨分隔所有同為 user mode 的行程，還要有位址空間與個別授權。

靜態 domain 把各階段需求聯集：compile 需讀source、link需寫binary，若全程持兩權就擴大每階段暴露。動態 domain 可在狀態切換時改權限，但切換本身也必須授權、保存/還原正確 caller context，並檢查參數。

傳統 UNIX setuid 執行檔可在成功 exec 的指定條件下改 effective UID 為檔案 owner；不是把 real UID 一律改掉，也不是單純呼叫一般函式就改 UID。例 realUID=1001、owner=0：執行受允許的 setuid 程式後 euid 可為0，realUID仍1001；程式需正確降權、驗證環境與輸入。具體抑制條件依 OS，這裡不把歷史機制當成任意提權方法。

MULTICS rings：受控進入，不是內圈任意入口

書中歷史抽象模型：ring0較高權；access bracket (b1,b2)，gate limit b3≥b2；caller ring i 在 [b1,b2] 可直接執行。i>b2 時，只有 i≤b3 且入口是指定 gate 才可轉入較高權；i<b1 轉往較低權時，參數不得讓低權程式任意引用其不可見區域。

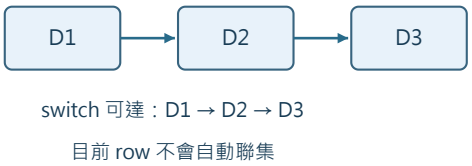
例 bracket=[1,3]、limit=5、gate={entryA}：i=2 呼叫一般入口可保持ring2；i=4 呼叫entryA可受控進入ring3，呼叫entryB拒絕；i=6即使entryA也拒絕。返回要恢復 caller ring/stack。此例限於書中 ring model；完整 MULTICS 還有 read/write brackets 等硬體細節。

純巢狀權限無法表達互不包含的需求：D甲只需F、D乙只需G，若強制D甲⊆D乙或反之，至少一方會多得不需的物件。一般 access matrix 能表達這種非階層分隔。

同一 access matrix 的初值

授權矩陣與 domain-switch 可達性

domain	F	G	P	D2	D3
D1	r*	ø	ø	switch	ø
D2	w	r	ø	ø	switch
D3	ø	ø	print	ø	ø



switch 可達：D1 → D2 → D3

目前 row 不會自動聯集

r* = read + 同物件授權傳播；ø = 沒有權利。右圖只表示 switch，不代表直接入口任意可用。

令 A[D,O] 是操作集合；空格就是∅，不表示「尚未決定」。目前process只使用當前domain那一row。D1可讀F，不能寫F；D2可寫F且可切D3；D3可印P。D1能切D2再切D3，代表可能取得後續domain的行為能力，不能只看D1原始row就斷言它永遠無法印P。

domain-switch 本身當作對 domain object 的 switch 操作。切換後不是把兩row聯集；若需要受控服務，還應限制入口與參數，避免任意直接進入高權程式中段。

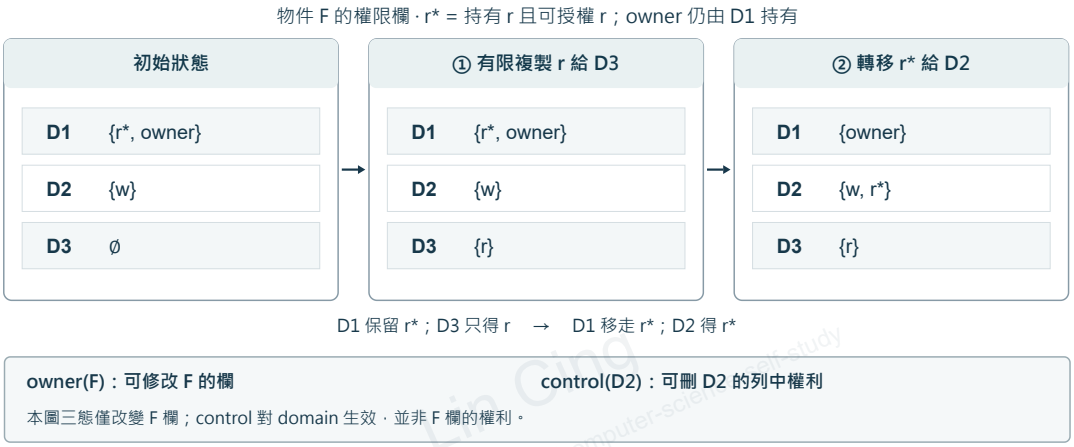
copy、transfer、limited copy、owner、control

模型中的 r* 表示持有 r，且可在**同一 object column** 傳播 r；不能用 F 的 r* 給 G 加 r。普通 copy 保留來源；transfer 將本例的 r* 從來源移到目標；limited copy 僅給 r，目標不獲傳播權。

獨立初態。 此例不是前一張矩陣的延續：A[D1,F]={r*, owner}、A[D2,F]={w}、A[D3,F]=∅。D1 對 D3 執行 limited copy 後，D3={r}，不能靠這項權利再轉授。D1 作為 F 的 owner，可在 F 的 column 中增加或刪除指定權利，例如刪除 D2 的 w；owner(F) 不代表擁有所有物件。

control 是對 domain object 的權利。若 control∈A[D1,D2]，本模型允許 D1 從 D2 的 row 移除權利，不允許任意新增。刪除 D2 對 D3 的 switch，會中斷原來 D1→D2→D3 的路徑；其他路徑是否存在須另行檢查。

用同一欄追蹤授權的複製與轉移



此例使用獨立初始狀態：D1 持有 F 的 `r*` 與 `owner`，D2 持有 `w`。有限複製只給 D3 一般 `r`；轉移再把 D1 的 `r*` 移給 D2。`owner(F)` 管 F 欄；另有 `control(D2)` 時，才可依模型刪除 D2 列的權利。

保護權利不等於禁止資訊流

即使D3只有read(F)且沒有copy(F)權利，只要它能read資料並write(G)，就可把內容逐byte寫到G。copy right限制的是授權的傳播，不是資訊的複製。已讀進user記憶體的資料通常也不能透過撤銷F的read「收回」。

可用不變量證明受限系統安全：若唯一授權操作是同column、只能授出來源已有權利的attenuation，且沒有owner/建立新權利規則，則任一後繼狀態的權利都源自初始某個domain對同object持有的權利。歸納：初始成立；每一步新增r必有來源r，沿來源鏈回溯到初始。此結論不保證機密資訊不外流，也不能套到允許owner任意造新權利的模型。

稀疏矩陣的四種實作

表示	索引方向	回答什麼最快	更新/保護責任
global triples (D,O,R)	全域或雙索引	任一pair	保護全域表，避免每次線性掃全表
ACL(O)=[(D,R)]	object column	誰可用O	可集中刪O的授權；須定義matching/default規則
capability list(D)	domain row	D目前持有哪些能力	cap不可偽造；傳遞/attenuation由機制控制
lock/key	object locks與domain keys	是否有matching key	key/lock清單本身受保護，換key可撤銷

capability 是受保護的物件參考＋權利；數字fd是本行程表的索引，不能靠猜別人的fd就取得其capability。不可偽造可由kernel table、tagged hardware或適當的受保護表示達成；不是「所有capability都必須永遠不能出現在user位址空間」。

同一權限的 row／column 轉換與檢查

F的ACL=[D1:r,D2:rw]，G的ACL=[D2:r,D3:w]。轉成cap lists：D1=[F:r]；D2=[F:rw,G:r]；D3=[G:w]。兩者在同一時點表示相同矩陣，但委派/撤銷的更新成本不同。若採hash索引查pair可期望O(1)，線性list查長k則O(k)，不能把表示名稱直接當作固定O(1)。

hybrid：open時檢ACL，給kernel保存的handle{F,read};後續write查mode拒絕。若此後ACL刪read，舊handle是否仍可read依撤銷政策；「改ACL必立刻撤所有已開啟fd」並非UNIX的一般保證。新增每次generation檢查可改此語義，但需處理mapping與in-flight操作。

教學ACL採只有allow且union/default deny；真系統若有deny ACE/順序/群組規則，需要另一套明確求值演算法，不能把此簡化算法套到Windows DACL。

RBAC：user→role→權限與任務的映射

歷史案例固定為 Solaris 10。其RBAC把roles、rights profiles、authorizations與process privileges分開：role聚合管理任務，profile可指定命令與執行屬性，authorization可由特定應用檢查，privilege由kernel對操作檢查。不是所有角色權利都只是某個UID，也不是authorization字串本身直接替代kernel privilege。

原創模型：user U可assume BackupRole；該role可執行BackupCmd，執行時只帶所需讀取/目標寫入權；不能改GradeDB。user V可assume GradeRole，但不能替自己核准發布。啟用角色的認證與稽核要另做，單看user-to-role配置不能推出所有請求均允許。

形式化簡化RBAC：UA⊆Users×Roles、PA⊆Roles×Permissions、session activeRoles⊆assignedRoles(U)，當且僅當某active role含所需permission才允許。若有互斥roles，啟用時再驗constraints。此式是教學RBAC模型，不等同Solaris全部privilege集合運算。

撤銷有四個獨立維度

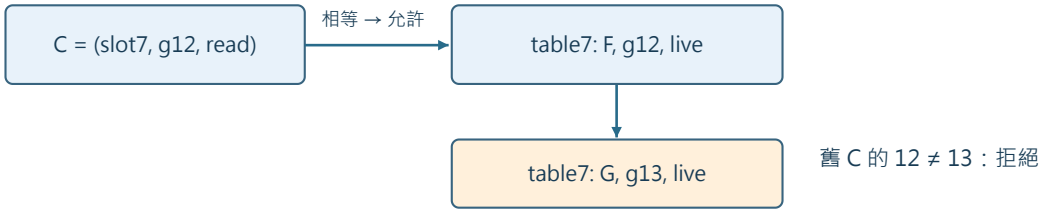
immediate/delayed 表示何時生效；selective/general 表示影響哪些持有人；partial/total 表示撤銷哪些操作；temporary/permanent 表示未來是否能再授權。「即刻」仍須定義邊界：阻止之後的新操作，不代表已開始的 I/O 自動取消，也不能收回已讀出的資料。

ACL 可集中刪除 entry，但已核發的 handle 或快取可能延後效果。分散的 capability 可採重新取得（reacquisition）／租期、back-pointers、indirection 或 key/version；back-pointers 要追蹤全部複本，indirection 則把查驗收斂到可撤銷的 slot。更換共用 master key 會撤銷使用該 key 的所有能力；選擇性撤銷需要分組 key 或代理。

租期範例。 使用一致且可信的時間基準；授權從 t=0 起有效至 t=10 秒，且每次新操作都先檢查未過期。t=3 撤銷後，授權者拒絕續租；即使撤銷通知未立即送達，t≥10 的新操作也因租期到期而拒絕，最長延遲為 7 秒。若允許無限快取、過期後使用或撤銷後續租，便不能聲稱這個延遲上限。已在途操作是否完成，由另外的取消契約決定。

Indirection 撤銷：generation 避免 ABA

同一 slot 重用：不能只比較索引 7



撤銷 / 重用時換 generation；resolve 的檢查與取得物件 reference 必須同步。

cap C={slot=7,generation=12,rights=r}；table[7]={obj=F,g=12,live=true}。每次resolve先驗live與generation相等，再查r，並在同一同步保護下取得obj存活reference。撤銷設live=false；重用slot存G時generation升13。舊C仍是(7,12)，對(7,13)拒絕，不能錯指G。

一般證明條件： 不重用仍可能存在的generation，且check→取得穩定reference不可被撤銷/重配插入。則接受C必對應它核發時的物件版本。若generation可wrap且舊C永存，終會ABA；必須不重用或先確保舊tokens全消失。若只驗slot數字，F撤銷後G占slot7就會讓舊cap取得G，是確定的權限漏洞。

Hydra：抽象操作與受限 amplification

歷史Hydra讓user-defined type有auxiliary rights。原創例 Account物件A的client cap只含deposit，不含代表資料的raw read/write。呼叫受信任deposit(A,10)時，機制確認型別與操作權，受保護procedure暫時獲得實作A所需的representation access，更新balance=40→50；返回後client仍只有deposit。

amplification不是「caller自選要升權」；只由已認可的procedure、指定型別與受限規則觸發。若任意routine都能把readonly參數升成write，就破壞caller的保護需求。因此互不信任subsystems需要控制可放大的權利、可保存的參考及參數範圍；不能只說封裝就自動安全。

Cambridge CAP：資料能力與軟體能力

data capability由machine/microcode強制read/write/execute段權限；software capability的自定義意義由受保護procedure解釋，底層保證不能任意偽造/篡改。seal/unseal式權限讓只有對應型別與環境的procedure能檢視受保護表示。

與Hydra共同點：client只拿抽象操作能力，implementation可能需要更低層存取；邊界是受保護的typed procedure。差別是權利解釋與支援庫的設計，不是「CAP能執行任何user程式就一定安全」。若Bank子系統的procedure錯把balance寫壞，Bank語義仍壞；隔離只把損害限制在其保護環境，不能證明Bank業務不變量。

本節按原書歷史架構比較，Account數值為原創抽象模型，不冒充Hydra/CAP可直接執行的原始API。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

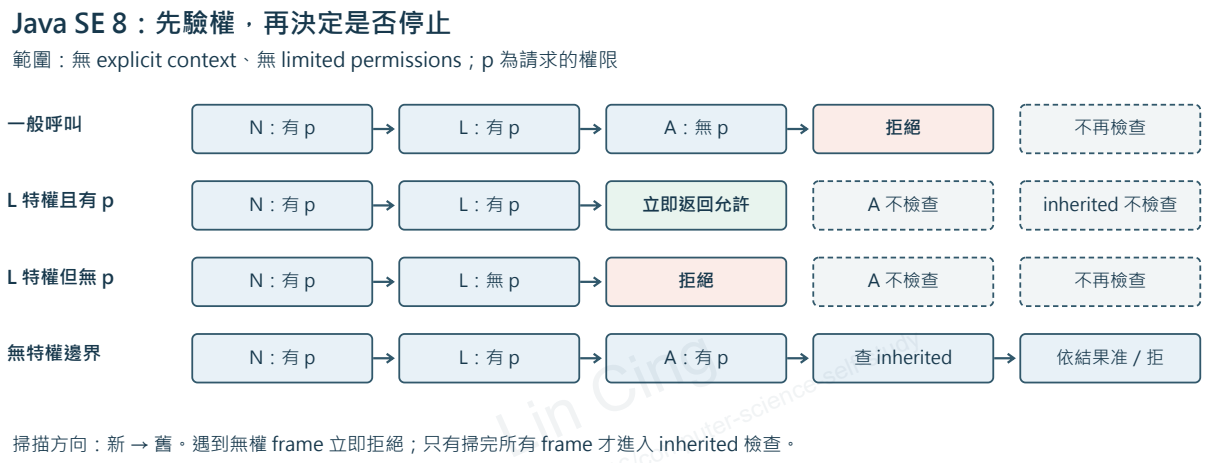
編譯期、load-time、run-time 各能保證什麼

language把存取宣告放在type/module/interface邊界；compiler可靜態排除非法操作，runtime檢查動態index/type/capability。若執行者可改機器碼、偽造reference或透過不受管native入口繞過檢查，單靠原始碼private欄位不足以隔離敵對程式。

type safety保證類型/記憶體規則，不代表業務授權。能合法取得Account reference的程式，仍可能錯誤地允許任意提款；type-correct也可能資訊外洩。kernel隔離粗到process，language可細到同process內的class/object，但其runtime、loader、native interface進入trusted computing base。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Java SE 8 的 stack inspection：privileged 也先驗權



限 Java SE 8 傳統模型，doPrivileged 不帶 explicit context 或 limited permissions。由新到舊 N→L→A，p=connect(proxy.example:80)；N、L 有 p，A 沒有。普通檢查在 A 拒絕。若 L 使用 doPrivileged，N、L 均通過後立即允許，A 與 inherited context 不再檢查；若 L 無 p，仍在 L 拒絕。

只有沒有命中提前返回的 privileged 邊界，且所有 frame 均通過時，才檢查 thread 的 inherited context。帶其他參數的版本另有規則；本例不是現代 Java 沙箱建議。

Confused deputy：在高權框架驗證實際目標

Loader為了讓低權Applet讀取代理服務，可在受控privileged區塊連到固定proxy。若Loader把Applet給的任意addr原封不動放入doPrivileged，會讓低權caller借Loader的網路權掃內網。正確例先把目的地固定在已驗證的proxy、限制操作與回傳資料，不把整個user callback都納入高權區塊。

stack inspection的歸納條件：每個已檢查的frame都具有p；遇到privileged邊界時，其本身也具有p才停止。由此可保證「本次授權不超過邊界caller已有的p」，但仍須信任邊界caller選擇了正確object/參數，不能據此證明整體服務政策正確。

從矩陣到實作的等價條件

access matrix只指定當前允許關係；ACL/capabilities是不同索引，dynamic operations再改關係。實作要保存object identity、right的型別與domain context，並確保不可偽造、不可繞過。若查驗結果被cache，必須把撤銷語義一起定義。

owner操作column，control操作domain row的刪除；switch改目前domain；copy/transfer/limited-copy依規則改同object授權。這些操作的組合會改未來可達domain，所以單看靜態row不能推出整段程式的權限上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

自學時可檢驗的三類結論

①授權安全：每次操作是否有相應權利；②狀態安全：撤銷、重用、競爭後是否錯指物件；③資訊/業務安全：即使所有權限檢查都通過，是否仍可藉合法read/write外洩或濫用高權服務。三者需要不同證據。

一般證明必須先限定允許的狀態轉移。cap attenuation的歸納不適用於有任意owner加權的系統；generation證明不適用於無界舊token搭配有限wrap；Java privileged邊界也不免除參數驗證。練習以完整矩陣、slot版本與call stack把這些界線具體化。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz et al., Operating System Concepts, 9th ed., Ch. 14. Oracle: Java SE 8 AccessController. Oracle: Solaris 10 RBAC.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study